MANGOBOOST

AMDSOW InferenceX user guide

Operator workflow and manual Slurm runbook

AMDSOW InferenceX DeepSeek-R1-0528 FP8 MI300X vLLM PD-disaggregation validation package

Confidential. Shared with AMD under NDA. | contact@mangoboost.io

Document control

Field	Value
Document	AMDSOW InferenceX user guide
Version	v1.1.0
Last updated	2026-06-27
Owner	MangoBoost AMDSOW delivery team
Contact	contact@mangoboost.io
Audience	AMD/customer operators and solution engineers
Status	Generated A4 PDF from maintained AMDSOW InferenceX Markdown documentation
Source format	Maintained Markdown rendered through HTML/CSS with Python-Markdown and WeasyPrint.

Table of contents

AMDSOW InferenceX user guide

This guide is for the operator who needs to run the AMDSOW DeepSeek-R1-0528 FP8 MI300X vLLM prefill/decode benchmark through **InferenceX**.

In plain terms: you select one checked-in YAML row. GitHub Actions starts the run. A self-hosted runner submits a Slurm job. Slurm allocates MI300X GPU nodes. Docker starts vLLM prefill and decode workers. The benchmark client sends requests and writes JSON results.

Use the GitHub Actions path first. Use the manual Slurm path only when Actions is unavailable or you are debugging directly from a Slurm-capable host.

Fresh-eye quick start

Use this path if you are opening the repository for the first time and just need one clean validation run.

1. Confirm you are in the repo root:

```
cd /mnt/shared_mango/amdsow-deliveries/regress/amdsow-inferencex-pr
git branch --show-current
```

1. Decide which run path you are using:

Situation	Use this path
You have GitHub Actions access and want the normal operator path	Sections 6-9
You are on a Slurm-capable host and want direct control/debugging	Section 10
You need the full 20-row best-config regression	Section 10, "Run the full 20-row best-config regression"

1. For the first smoke test, run `8k1k c256`. It is the default row and uses 3 Slurm nodes.
2. If you are on a Slurm-capable host, check the local queue before submitting:

```
squeue -u "$USER" -o '%.18i %.30j %.8T %.20R'
sinfo -p compute -N -o '%N %t %E'
```

Skip this step when you are only dispatching through GitHub Actions from a non-cluster workstation.

1. After a manual Slurm job finishes, summarize the exact log directory from that run:

```
python3 utils/summarize_slurm_results.py "$BENCHMARK_LOGS_DIR"
```

Expected clean end state:

- Slurm job state is `COMPLETED`.
- The log directory contains `slurm_job-<job-id>.out` and `slurm_job-<job-id>.err`.
- The summary script prints rows with `State` = `DONE`.
- GSM8K is in the expected mid-90% range for the current MI300X DSR1 setup.

Do not edit YAML, pin nodes, or change Docker images for a first run. Use the checked-in config and the production image first; change one variable at a time only when debugging.

1. What you are running

Field	Value
Repo path in this guide	`/mnt/shared_mango/amdsow-deliveries/regress/amdsow-inferencex-pr`
Repo slug (`origin`)	`amdsow/InferenceX`
Workflow	`.github/workflows/e2e-tests.yml`
Config file	`.github/configs/amd-master.yaml`
Config key	`dsr1-fp8-mi300x-vllm-disagg`
Model	`deepseek-ai/DeepSeek-R1-0528`
Runtime image	`docker.io/chaeminlimmb/vllm-mori-pd:milestone4-aiterwheel`
Hardware	AMD MI300X, 8 GPUs per node
Framework	`vllm-disagg`
Runner label/type	`mi300x-disagg`
Default row	`8k1k`, concurrency `256`

PD disaggregation means the server role is split:

Role	What it does	In this default row
Prefill	Reads the prompt and builds KV cache.	2 TP8 prefill workers on 2 nodes
Decode	Generates output tokens using the transferred KV cache.	1 DP8EP decode worker on 1 node

The default row uses 3 Slurm compute nodes total. It disables MTP speculative decoding (`spec-decoding: none` , `DECODE_MTP_SIZE=0`) and sets both PD roles to vLLM block size 1 through `PREFILL_BLOCK_SIZE=1` and `DECODE_BLOCK_SIZE=1` .

There is no `8k1k c512` row in this repository revision. Use `8k1k c256` unless the config owner adds a new row.

2. Names that are easy to mix up

Term	What it is	Example
GitHub runner label/type	The `runs-on` target selected by the workflow. It chooses a pool of self-hosted runners.	`mi300x-disagg`
GitHub runner name	The specific self-hosted runner machine that picked up the job. The launcher name is derived from it.	`mi300x-amds_06`
Slurm compute node	A physical MI300X host allocated by Slurm for the job. Discover these at run time.	site-specific, not committed in docs
Benchmark worker	One vLLM server role process: prefill or decode.	`2P1D` = 2 prefill workers, 1 decode worker
TP8	Tensor-parallel vLLM profile across 8 GPUs.	TP8 prefill
DP8EP	Wide expert-parallel vLLM profile: data parallel 8 plus expert parallel.	DP8EP decode
MTP	Multi-token speculative decoding.	Disabled in c256 (`MTP0`)

Do not hard-code Slurm compute node names in docs, YAML, or scripts. If you need fixed nodes for debugging, discover currently idle nodes with Slurm and pass `NODELIST` only for that one run.

3. Slurm basics for this workflow

A Slurm job is one scheduled unit of work. Here it means one `sbatch` allocation that reserves MI300X nodes and runs `job.slurm` across those nodes.

This repo's Slurm path has four parts:

Part	Meaning in this repo
Allocation request	`submit.sh` calls `sbatch --exclusive -N <nodes> -n <nodes> --partition compute` .
Job script	`benchmarks/multi_node/amd_utils/job.slurm` . It validates the model, starts Docker containers, and starts the router/proxy.
Runtime processes	Docker containers running `server_vllm.sh` , vLLM prefill/ decode workers, MoRIIO proxy, and benchmark client.
Outputs	Slurm `.out/.err` , per-concurrency JSON, optional per-node vLLM logs, GitHub artifacts.

Common Slurm states:

State	Meaning	Operator action
`PD` / `PENDING`	Waiting for nodes or policy.	Check `sinfo` and queue reason.
`R` / `RUNNING`	Allocation is active.	Tail logs and wait for vLLM health / benchmark output.
`CG` / `COMPLETING`	Processes are exiting and Slurm is collecting status.	Wait unless it hangs for a long time.
`CD` / `COMPLETED`	Job exited 0.	Download artifacts / read metrics.
`F` / `FAILED`	Job exited non-zero.	Read Slurm output and failed workflow logs.
`CA` / `CANCELLED`	Someone cancelled it.	Confirm this was intentional.

Useful commands from a Slurm-capable host:

```
squeue -u "$USER" -o '%.18i %.30j %.8T %.20R'
sinfo -p compute -N -o '%N %t %E'
scontrol show job <job-id>
scancel <job-id>
```

`submit.sh` rejects a pinned node list unless the host count equals `PREFILL_NODES + DECODE_NODES`. The default `8k1k c256` row needs exactly 3 nodes.

4. Prerequisites

Run commands from the repo root:

```
cd /mnt/shared_mango/amdsow-deliveries/regress/amdsow-inferencex-pr
```

Local tools on the dispatch machine:

- `git`;
- `.venv/bin/python` with the InferenceX matrix dependencies;
- `gh`, authenticated against `amdsow/InferenceX`, if using GitHub Actions;
- Slurm commands (`squeue`, `sinfo`, `scontrol`, `sbatch`, `srun`) only for the manual cluster path.

Create the venv if it is missing:

```
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install pydantic pyyaml tabulate
```

GitHub and cluster requirements:

Requirement	Why
A self-hosted runner with label `mi300x-disagg`, online and idle	The workflow targets this label, and that runner must reach the MI300X Slurm cluster.
Actions secret `REPO_PAT`	Used by the workflow checkout/upload path.
Actions secret `INFERENCEX_OFFICIAL_RO_HF_TOKEN`	Exported as `HF_TOKEN`; this recipe still expects staged weights.
Slurm partition `compute`	The MI300X launcher submits there.
Model weights staged on every allocated node	vLLM needs the model before serving starts.

The vLLM branch searches the model root for `DeepSeek-R1-0528`. If it cannot find the weights, look for this error in Slurm output:

```
FATAL: Model 'DeepSeek-R1-0528' not found. Searched:
```

The run then prints the host paths it checked. Fix model staging before rerunning.

5. Choose a row

The config key `dsr1-fp8-mi300x-vllm-disagg` defines these valid concurrencies:

Sequence length	ISL / OSL	Valid concurrencies
`1k1k`	1024 / 1024	1, 6, 9, 30, 60, 117, 231, 462, 615, 1229
`8k1k`	8192 / 1024	1, 2, 6, 9, 16, 24, 30, 77, 154, 256

Start with `8k1k --conc 256`. It is the current mixed best-config row:

Field	Value
Topology	`2P1D` : 2 prefill workers, 1 decode worker
Nodes	3 total: 2 prefill nodes + 1 decode node
Prefill profile	TP8 with `PREFILL_BLOCK_SIZE=1` override
Decode profile	DP8EP with `DECODE_BLOCK_SIZE=1`
MTP	Off (`DECODE_MTP_SIZE=0`)
Benchmark pressure	concurrency 256, ISL 8192, OSL 1024

A value outside the valid list matches no row. Validate before dispatching.

6. Validate the row locally

This step uses no GPUs. It expands the YAML and confirms you selected the intended row.

```
CONFIG_JSON=$(.venv/bin/python utils/matrix_logic/generate_sweep_configs.py test-config \
  --config-files .github/configs/amd-master.yaml \
  --config-keys dsr1-fp8-mi300x-vllm-disagg \
  --seq-lens 8k1k --conc 256 --no-evals)

echo "$CONFIG_JSON" | .venv/bin/python -c '
import sys, json
rows = json.load(sys.stdin)
if not rows:
    sys.exit("EMPTY: no row matched. Check --seq-lens and --conc against the matrix in section 5.")
for r in rows:
    print("isl/osl :", r.get("isl"), "/", r.get("osl"))
    print("conc    :", r.get("conc"))
    print("spec    :", r.get("spec-decoding"))
    print("runner  :", r.get("runner"))
    print("prefill :", r.get("prefill"))
    print("decode  :", r.get("decode"))
    print()
print(len(rows), "row(s)")
'
```

For `8k1k --conc 256` expect one row with:

- `isl/osl` : 8192 / 1024;
- `conc` : `[256]` ;
- `spec` : `none` ;
- `runner` : `mi300x-disagg` ;
- prefill `additional-settings` : `PREFILL_NODES=2` , `PREFILL_BLOCK_SIZE=1` ;
- decode `additional-settings` : `DECODE_NODES=1` , `DECODE_DP8EP=true` , `DECODE_BLOCK_SIZE=1` , `DECODE_MTP_SIZE=0` .

Pretty-print the full JSON if you need to inspect every field:

```
echo "$CONFIG_JSON" | .venv/bin/python -m json.tool
```

Validate from the same ref you will dispatch. If `REF_UNDER_TEST` is non-empty in section 7, run this local validation from that branch or SHA before dispatching.

7. Dispatch one row through GitHub Actions

Set the repo, workflow branch, optional code-under-test ref, and a unique run name:

```
export REPO="amdsow/InferenceX"
export WORKFLOW_REF="main"
```

```
export REF_UNDER_TEST=""  # Optional: branch or SHA to test. Leave empty to use WORKFLOW_REF's commit.
export TEST_NAME="DSR1 MI300X vllm-disagg 8k1k c256 $(date -u +%Y%m%dT%H%M%SZ)"
```

`WORKFLOW_REF` is the branch where `.github/workflows/e2e-tests.yml` is read from. `REF_UNDER_TEST` is the code that Actions checks out to generate configs and run the benchmark. Leave it empty unless you intentionally test a branch or SHA.

Before dispatching, verify the shared `mi300x-disagg` runner and target Slurm cluster are idle using the site-approved status source. If you are on a Slurm login host, `squeue` is only an additional check; it may not show jobs owned by the GitHub runner account or other operators.

Dispatch the default row:

```
gh api -X POST \
  "/repos/${REPO}/actions/workflows/e2e-tests.yml/dispatches" \
  -f "ref=${WORKFLOW_REF}" \
  -f "inputs[ref]=${REF_UNDER_TEST}" \
  -f "inputs[test-name]=${TEST_NAME}" \
  -f 'inputs[generate-cli-command]=test-config --config-files .github/configs/amd-master.yaml --config-keys
dsr1-fp8-mi300x-vllm-disagg --seq-lens 8k1k --conc 256 --no-evals' \
  -f 'inputs[duration-override]='
```

The `generate-cli-command` value is the generator command from section 6 without the `.venv/bin/python` prefix. Keep `--no-evals` for throughput-only validation. Drop it only when you intentionally run evals.

8. Find and watch the GitHub run

The dispatch API does not return a run id. Find the run by its unique `TEST_NAME`:

```
sleep 8

RUN_ID=$(gh run list --repo "$REPO" \
  --workflow e2e-tests.yml \
  --event workflow_dispatch \
  --limit 30 \
  --json databaseId,displayTitle,createdAt \
  --jq "map(select(.displayTitle == \"e2e Test - ${TEST_NAME}\")) | sort_by(.createdAt) | last
| .databaseId")

if [ -z "$RUN_ID" ] || [ "$RUN_ID" = "null" ]; then
  echo "No matching run yet. Wait a few seconds and run this block again."
else
  echo "RUN_ID=$RUN_ID"
fi
```

Watch it to completion:

```
gh run watch "$RUN_ID" --repo "$REPO" --exit-status
```

If it fails, read the failed step logs:

```
gh run view "$RUN_ID" --repo "$REPO" --log-failed
```

9. Download and read artifacts

Start with a clean output directory so old JSON does not pollute the summary:

```
rm -rf artifacts
mkdir -p artifacts

gh run download "$RUN_ID" --repo "$REPO" --pattern 'bmk_*' --dir artifacts
gh run download "$RUN_ID" --repo "$REPO" --pattern 'multinode_server_logs_*' --dir artifacts || true
```

```
gh run download "$RUN_ID" --repo "$REPO" -n results_bmk --dir artifacts
gh run download "$RUN_ID" --repo "$REPO" -n run-stats --dir artifacts
```

Artifact	Contents
`bmk_<RESULT_FILENAME>`	Per-config processed benchmark JSON (`agg_<RESULT_FILENAME>_*.json`).
`multinode_server_logs_<RESULT_FILENAME>`	Present only when the launcher produced `multinode_server_logs.tar.gz`; the MI300X launcher usually does not.
`results_bmk`	`agg_bmk.json`, aggregated across the run.
`run-stats`	`run_stats.json`, with job/node status and computed success rate.

Summarize metrics:

```
.venv/bin/python utils/summarize.py artifacts
```

A run is successful when all of these are true:

1. `gh run watch` exits 0.
2. `gh run download` retrieves a `bmk_*` artifact.
3. `utils/summarize.py artifacts` prints one non-empty row for `8k1k c256`.
4. `run-stats/run_stats.json` reports the benchmark job as successful.

There are no checked-in AMDSOW performance thresholds for these rows. Treat the first clean run as the baseline unless the delivery owner gives target numbers.

10. Manual Slurm path

Use this only from a host that can submit to the target Slurm cluster. It calls the same InferenceX launcher path that GitHub Actions uses.

Check the queue and node state first:

```
cd /mnt/shared_mango/amdsow-deliveries/regress/amdsow-inferencex-pr
squeue -u "$USER" -o '%.18i %.30j %.8T %.20R'
sinfo -p compute -N -o '%N %t %E'
```

Prepare the default `8k1k c256` environment:

```
export EXP_NAME="dsr1_8k1k" ISL=8192 OSL=1024 CONC_LIST="256" SPEC_DECODING="none"
export PREFILL_NUM_WORKERS=2 PREFILL_TP=8 PREFILL_EP=1 PREFILL_DP_ATTN=false PREFILL_NODES=2
PREFILL_DP8EP=false PREFILL_BLOCK_SIZE=1
export DECODE_NUM_WORKERS=1 DECODE_TP=8 DECODE_EP=8 DECODE_DP_ATTN=false DECODE_NODES=1 DECODE_DP8EP=true
DECODE_BLOCK_SIZE=1 DECODE_MTP_SIZE=0

export MODEL="deepseek-ai/DeepSeek-R1-0528" MODEL_PREFIX="dsr1" PRECISION=fp8 FRAMEWORK="vllm-disagg"
export IMAGE="docker.io/chaeminlimmb/vllm-mori-pd:milestone4-aiterwheel" RANDOM_RANGE_RATIO=0.8
export IS_MULTINODE=true KEEP_LOGS=1
export RUN_EVAL=true EVAL_ONLY=false EVAL_SERVER_MAX_MODEL_LEN=20480 EVAL_SERVER_BLOCK_SIZE=1
unset EVAL_CONC
export RUNNER_NAME="amdsow-dsr1-c256-$(date -u +%Y%m%dT%H%M%SZ)" RUNNER_TYPE="mi300x-disagg"
export GITHUB_WORKSPACE="$PWD" BENCHMARK_LOGS_DIR="$PWD/benchmark_logs"
export AMDSOW_SLURM_EXCLUDE_NODES="${AMDSOW_SLURM_EXCLUDE_NODES:-a05u43,a04u43}"

# Optional. Preserve raw per-node vLLM logs on shared storage.
# export VLLM_LOG_ARCHIVE_DIR="$PWD/vllm_logs_8k1k_c256_$(date -u +%Y%m%dT%H%M%SZ)"
export RESULT_FILENAME="validation_8k1k_c256"

# Optional. Pin exactly 3 idle nodes discovered with sinfo/squeue. Leave unset to let Slurm choose.
# export NODELIST="<node-a>,<node-b>,<node-c>"
```

`BENCHMARK_LOGS_DIR` must be on shared storage visible from the submit host and allocated Slurm nodes. `$PWD/benchmark_logs` is safe only when this repo path is shared across the cluster.

Run a no-allocation dry run before the real submit:

```
SUBMIT_DRY_RUN=1 bash runners/launch_mi300x-amds.sh
```

The dry run should print `SUBMIT_DRY_RUN=1: not calling sbatch` and a resolved `sbatch` command. It should also show the intended node count and exported env. Fix env mistakes before continuing.

Launch detached and follow the launcher log:

```
setsid bash runners/launch_mi300x-amds.sh </dev/null >/tmp/run_8k1k_c256.log 2>&1 &
tail -f /tmp/run_8k1k_c256.log
```

When the job completes, print a compact result table from the Slurm log directory:

```
python3 utils/summarize_slurm_results.py "$BENCHMARK_LOGS_DIR"
```

The table columns are:

Column	Meaning
`Row`	Sequence length plus concurrency, for example `8k1k_c256`.
`State`	`DONE` when the script found a completed benchmark result block.
`Total`	Total token throughput per GPU.
`Gen`	Output token throughput per decode GPU.
`Interactivity`	`1000 / mean TPOT ms`; higher is better.
`E2E`	Mean end-to-end request latency.
`Cost`	Relative cost score using the default `390 / Total` convention.
`GSM8K strict/flex`	lm-eval accuracy if `RUN_EVAL=true`; otherwise `-`.

Run the full 20-row best-config regression

Use this block when you need the full published best-config set, not just the default `8k1k c256` smoke row. It submits 9 Slurm jobs that cover all 20 datapoints:

Job label	Datapoints
`1k1k_c1_c117`	`1k1k` conc `1,6,9,30,60,117`
`1k1k_c231`	`1k1k` conc `231`
`1k1k_c462_c1229`	`1k1k` conc `462,615,1229`
`8k1k_c1`	`8k1k` conc `1`
`8k1k_c2`	`8k1k` conc `2`
`8k1k_c6_c30`	`8k1k` conc `6,9,16,24,30`
`8k1k_c77`	`8k1k` conc `77`
`8k1k_c154`	`8k1k` conc `154`
`8k1k_c256`	`8k1k` conc `256`

The command uses the current production image, `docker.io/chaeminlimmb/vllm-mori-pd:milestone4-aiterwheel`. Do not switch to experimental image tags for this regression. Eval concurrency defaults to 64 when `EVAL_CONC` is unset; set `EVAL_CONC` only when intentionally testing a different eval load.

```
cd /mnt/shared_mango/amdsow-deliveries/regress/amdsow-inferencex-pr

export MODEL="deepseek-ai/DeepSeek-R1-0528"
export MODEL_PREFIX="dsr1"
export PRECISION="fp8"
export FRAMEWORK="vllm-disagg"
export IMAGE="docker.io/chaeminlimmb/vllm-mori-pd:milestone4-aiterwheel"
export ROUTER_TYPE=moriio
export RANDOM_RANGE_RATIO=0.8
export IS_MULTINODE=true
export KEEP_LOGS=1
export RUNNER_TYPE="mi300x-disagg"
export GITHUB_WORKSPACE="$PWD"
export RUN_EVAL=true
export EVAL_ONLY=false
export EVAL_SERVER_MAX_MODEL_LEN=20480
export EVAL_SERVER_BLOCK_SIZE=1
unset EVAL_CONC

# Optional site-local exclusions.
export AMDSOW_SLURM_EXCLUDE_NODES="${AMDSOW_SLURM_EXCLUDE_NODES:-a05u43,a04u43}"
export RUN_TS="$(date -u +%Y%m%dT%H%M%SZ)"

submit_best() {
  local row="$1"
  export RUNNER_NAME="mi300x-best20-${row}-${RUN_TS}"
  export RESULT_FILENAME="validation_${row}"
  export BENCHMARK_LOGS_DIR="$PWD/benchmark_logs_best20_${row}_${RUN_TS}"
  setsid bash runners/launch_mi300x-amds.sh </dev/null >"/tmp/run_best20_${row}_${RUN_TS}.log" 2>&1 &
  echo "$row pid=$! launcher_log=/tmp/run_best20_${row}_${RUN_TS}.log logs=$BENCHMARK_LOGS_DIR"
  sleep 2
}

unset PREFILL_BLOCK_SIZE DECODE_BLOCK_SIZE
export EXP_NAME="dsr1_1k1k" ISL=1024 OSL=1024 SPEC_DECODING="mtp"
export PREFILL_NUM_WORKERS=1 PREFILL_TP=8 PREFILL_EP=1 PREFILL_DP_ATTN=false PREFILL_NODES=1
PREFILL_DP8EP=false
export DECODE_NUM_WORKERS=1 DECODE_TP=8 DECODE_EP=1 DECODE_DP_ATTN=false DECODE_NODES=1 DECODE_DP8EP=false
DECODE_MTP_SIZE=3
export CONC_LIST="1 6 9 30 60 117"; submit_best "1k1k_c1_c117"

export DECODE_MTP_SIZE=1
export CONC_LIST="231"; submit_best "1k1k_c231"

export PREFILL_TP=8 PREFILL_EP=8 PREFILL_DP_ATTN=true PREFILL_NODES=1 PREFILL_DP8EP=true
export DECODE_TP=8 DECODE_EP=8 DECODE_DP_ATTN=true DECODE_NODES=1 DECODE_DP8EP=true DECODE_MTP_SIZE=1
export CONC_LIST="462 615 1229"; submit_best "1k1k_c462_c1229"

unset PREFILL_BLOCK_SIZE DECODE_BLOCK_SIZE
export EXP_NAME="dsr1_8k1k" ISL=8192 OSL=1024 SPEC_DECODING="mtp"
export PREFILL_NUM_WORKERS=1 PREFILL_TP=8 PREFILL_EP=1 PREFILL_DP_ATTN=false PREFILL_NODES=1
PREFILL_DP8EP=false
export DECODE_NUM_WORKERS=1 DECODE_TP=8 DECODE_EP=1 DECODE_DP_ATTN=false DECODE_NODES=1 DECODE_DP8EP=false
DECODE_MTP_SIZE=4
export CONC_LIST="1"; submit_best "8k1k_c1"

export DECODE_NUM_WORKERS=2 DECODE_NODES=2 DECODE_MTP_SIZE=4
export CONC_LIST="2"; submit_best "8k1k_c2"

export DECODE_NUM_WORKERS=3 DECODE_NODES=3 DECODE_MTP_SIZE=3
export CONC_LIST="6 9 16 24 30"; submit_best "8k1k_c6_c30"

export DECODE_NUM_WORKERS=1 DECODE_NODES=1 DECODE_MTP_SIZE=3
export CONC_LIST="77"; submit_best "8k1k_c77"

export PREFILL_NUM_WORKERS=2 PREFILL_NODES=2
export DECODE_NUM_WORKERS=2 DECODE_NODES=2 DECODE_MTP_SIZE=3
export CONC_LIST="154"; submit_best "8k1k_c154"

export SPEC_DECODING="none"
export PREFILL_NUM_WORKERS=2 PREFILL_TP=8 PREFILL_EP=1 PREFILL_DP_ATTN=false PREFILL_NODES=2
PREFILL_DP8EP=false PREFILL_BLOCK_SIZE=1
export DECODE_NUM_WORKERS=1 DECODE_TP=8 DECODE_EP=8 DECODE_DP_ATTN=false DECODE_NODES=1 DECODE_DP8EP=true
DECODE_BLOCK_SIZE=1 DECODE_MTP_SIZE=0
export CONC_LIST="256"; submit_best "8k1k_c256"
```

For `1k1k_c462_c1229`, the matrix-facing env uses `PREFILL_TP=8`, `PREFILL_EP=8`, `PREFILL_DP8EP=true`, `DECODE_TP=8`, `DECODE_EP=8`, and `DECODE_DP8EP=true`. `server_vllm.sh` turns that DP8EP profile into runtime vLLM flags with `--tensor-parallel-size 1`, `--data-parallel-size 8`, `--enable-expert-parallel`, `--block-size 1`, `--max-model-len 20480`, and `--num-gpu-blocks-override 1372000` for eval.

Monitor the submitted jobs:

```
squeue -u "$USER" -o '%i %.52j %.10T %.20R %N %.12M %.12l'
tail -f benchmark_logs_best20_*/slurm_job-*.out
```

After all jobs reach `COMPLETED`, summarize only the new best20 log directories from that run:

```
python3 utils/summarize_slurm_results.py benchmark_logs_best20_*_"$RUN_TS"/
```

If you submitted with the block above and did not save the timestamp, this also works and picks the newest log per row from the paths you pass:

```
python3 utils/summarize_slurm_results.py benchmark_logs_best20_*/
```

Use the broad glob only when the workspace does not contain unrelated or stale best20 logs you do not want included.

Interrupting `tail -f` does not stop the launcher or Slurm allocation. Use these commands to monitor and cancel:

```
squeue -u "$USER" --name "$RUNNER_NAME" -o '%.18i %.30j %.8T %.20R'
tail -f "$BENCHMARK_LOGS_DIR"/slurm_job-*.out
scancel <job-id>
```

Manual-run notes:

- `EXP_NAME` must start with `dsr1_`. The launcher derives `benchmarks/multi_node/dsr1_fp8_mi300x_vllm-disagg.sh` from `EXP_NAME`, `PRECISION`, and `FRAMEWORK`.
- `RUNNER_NAME` becomes the Slurm job name. Use a unique value so `squeue --name "$RUNNER_NAME"` finds the right allocation.
- If you pin `NODELIST`, the host count must equal `PREFILL_NODES + DECODE_NODES`, which is 3 for c256.
- Results land at the repo root as `<RESULT_FILENAME>_*.json`.
- `KEEP_LOGS=1` keeps the top-level `BENCHMARK_LOGS_DIR` and `slurm_job-<id>.out/.err`, but the launcher still removes `BENCHMARK_LOGS_DIR/logs` after result copy. Set `VLLM_LOG_ARCHIVE_DIR` before launch to preserve per-node vLLM logs.

11. Troubleshooting by symptom

Symptom	Check
Dispatch returns 403	`gh auth status`, repo slug, token scope.
Run starts but no runner picks it up	The `mi300x-disagg` runner pool is offline or busy.
Generator prints `EMPTY`	`--seq-lens` or `--conc` is not in section 5.
Job fails before Slurm allocation	Secrets, `inputs[ref]`, runner access to Slurm.
Slurm job stays pending	`squeue` reason, partition availability, node drain state.
`FATAL: Model ... not found. Searched:`	Stage `DeepSeek-R1-0528` under the expected model root on every allocated node.
Benchmark artifact missing	Read `gh run view --log-failed`, then Slurm `.out/.err`.
Raw vLLM logs missing	Set `VLLM_LOG_ARCHIVE_DIR` before rerun; the default MI300X path does not preserve them reliably.
Metrics row is zero/empty	

Symptom	Check
	Confirm the local validation output matched c256, then run `python3 utils/summarize_slurm_results.py "$BENCHMARK_LOGS_DIR"`.
Summary script says no logs found	Pass the directory that contains `slurm_job-<id>.out`, not only the repo root.
Manual run keeps running after closing the terminal	Find it with `squeue --name "$RUNNER_NAME"` and cancel the Slurm job id.

12. Golden rules

1. Treat `.github/configs/amd-master.yaml` as the benchmark source of truth.
2. Validate locally before dispatching.
3. Keep GitHub runner labels, GitHub runner names, Slurm compute nodes, and benchmark workers separate.
4. Let Slurm pick nodes by default; pin `NODELIST` only for targeted debugging.
5. For the c256 row, carry both block-size overrides (`PREFILL_BLOCK_SIZE=1`, `DECODE_BLOCK_SIZE=1`) in manual runs so they match CI.
6. Preserve raw vLLM logs with `VLLM_LOG_ARCHIVE_DIR` if you may need failure analysis.
7. Do not use local validation wrappers as source of truth; use them only as optional conveniences around the checked-in YAML and launcher path.